오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

HTML/CSS

DAY 03

시맨틱 HTML과 웹 접근성 기초

같은 화면이라도 사람이 아닌 쪽이 읽을 때 차이가 납니다.
검색엔진, 화면낭독기, AI가 이해하는 문서를 만듭니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **시맨틱 태그로 페이지 영역을 나눌 수 있다**

   header, nav, main, section, article, footer로 문서를 구분합니다.

2. **제목 계층과 대체 텍스트를 규칙에 맞게 쓸 수 있다**

   h1부터 순서대로 내려가고, 이미지에는 목적에 맞는 alt를 적습니다.

3. **AI에게 구조 리뷰를 받고 검증할 수 있다**

   AI가 제안한 구조를 그대로 받지 않고 근거를 확인합니다.

오늘의 결과물

about-me.html — div 대신 시맨틱 태그로 영역을 나눈 개인 소개 페이지

ROADMAP

오늘의 흐름

01

왜 의미가 필요한가

읽는 쪽의 사정
표준과 지침

02

시맨틱 태그로 영역 나누기

여섯 개의 태그
실습 01

실습 포함

03

제목 계층과 대체 텍스트

h1~h6 설계
alt 작성 기준

04

폼 접근성과 AI 리뷰

label·fieldset
실습 02

실습 포함

앞의 두 단계에서 구조를 세우고, 뒤의 두 단계에서 세부를 다듬습니다.
오늘 정리한 구조는 1챕터 마지막 날인 Day6에서 다시 점검합니다.

되짚기 · 01

이틀 동안 만든 것

1	**Day1 · 문서 구조**	html·head·body와 제목, 문단, 목록, 이미지, 링크, 표
2	**Day1 · 결과물**	about-me.html — 읽기만 하는 개인 소개 페이지
3	**Day2 · 폼**	form의 action·method·name, input type, 기본 검증
4	**Day2 · 결과물**	signup.html — 값을 입력받아 보내는 회원가입 화면

화면은 갖춰졌습니다. 오늘은 그 안에 의미를 넣습니다

준비

오늘의 셋팅

```
Day03/
  about-me.html
  me.jpg
```

먼저 화면을 기억해 둡니다

- 브라우저에서 열어 지금 모습을 본다
- 바꾼 뒤에도 화면이 같아야 한다
- 한 영역씩 바꾸고 새로고침한다

오늘은 화면을 바꾸는 시간이 아니라 코드의 의미를 바꾸는 시간입니다

01

왜 의미가 필요한가

읽는 쪽의 사정 · 표준과 지침

01 · 출발점

화면은 같지만 코드는 다릅니다

div만 쓴 코드

```
<div class="top">
  <div class="menu">…</div>
</div>
<div class="body">…</div>
```

사람 눈에는 아무 문제가 없습니다.

시맨틱 태그를 쓴 코드

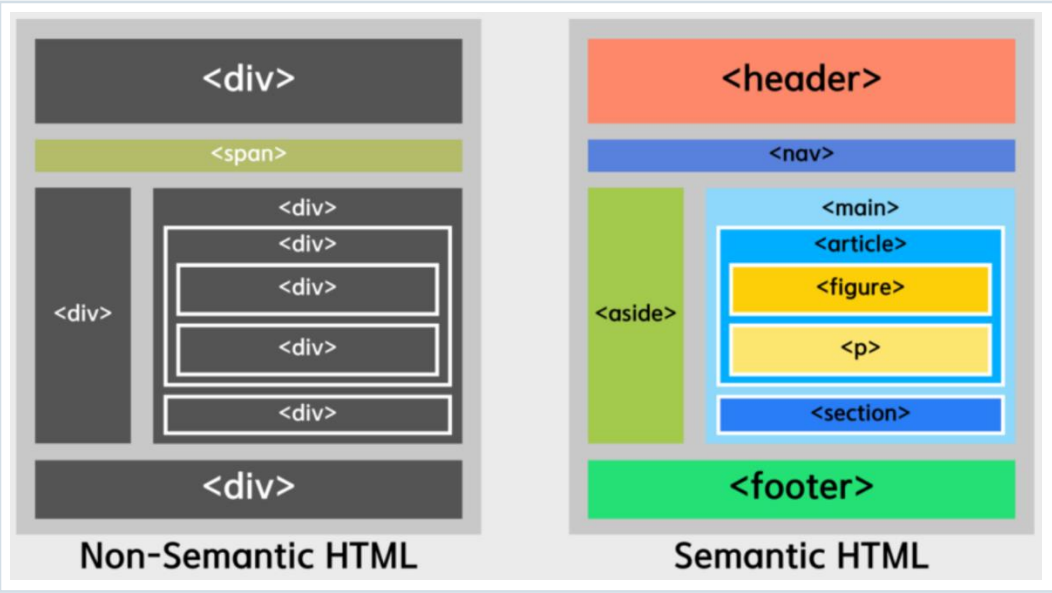
```
<header>
  <nav>…</nav>
</header>
<main>…</main>
```

기계가 영역을 구분할 수 있습니다.

두 코드는 브라우저 화면에서 똑같이 보입니다

차이는 검색엔진, 화면낭독기, AI처럼 사람이 아닌 쪽이 읽을 때 드러납니다.

더 정확한 의미를 제공합니다



왼쪽은 모두 div로 묶은 구조입니다.
오른쪽은 같은 자리에 의미 있는 태그를 쓴 구조입니다.

화면은 같고 코드가 다릅니다

01 · 읽는 쪽

Html을 읽는 다양한 주체들

검색엔진

페이지에서 무엇이 본문이고
무엇이 메뉴인지 구분합니다.
구조가 분명할수록
내용을 정확히 수집합니다.

화면낭독기

시각장애인이 쓰는 도구입니다.
영역 단위로 건너뛰며 읽는데,
div만 있으면 건너뛸 지점이
생기지 않습니다.

AI 도구

코드를 읽고 수정을 제안합니다.
영역이 구분돼 있으면
어디를 고쳐야 하는지
훨씬 정확하게 짚습니다.

셋 다 우리가 직접 볼 수 없는 독자입니다

그래서 화면만 보고 작업하면 문제가 있는지조차 알 수 없습니다.

검색엔진은 본문이 어디인지 찾습니다

```
<!-- 검색 결과에 쓰이는 정보 -->
<title>김코딩 – 웹 개발자</title>
<h1>김코딩</h1>
<main>
  <p>이 안의 글이 이 페이지의 내용입니다.</p>
</main>
```

메뉴와 본문을 구분합니다

nav 안의 글자는 어느 페이지에나
똑같이 들어 있습니다.
main 안의 글이 그 페이지만의
내용입니다.

구조를 나눠 두면 검색엔진이 페이지의 주제를 더 정확히 판단합니다

화면낭독기는 영역 단위로 건너뜁니다

```
<!-- 낭독기가 이렇게 읽습니다 -->
"배너, 김코딩"            ← header
"내비게이션, 링크 3개"    ← nav
"메인, 소개, 제목 수준 2" ← main

<!-- div 만 있으면 -->
"김코딩 소개 작업 연락처 …"  (그냥 이어서)
```

직접 들어볼 수 있습니다

macOS는 Command + F5로 VoiceOver를 켤 수 있고, Windows는 NVDA를 무료로 설치할 수 있습니다.

특별한 장비가 아니라 지금 쓰는 컴퓨터에 들어 있는 기능입니다

01 · AI 도구

AI도 구조를 보고 판단합니다

구조가 없는 코드를 주면

```
<div class="a">
  <div class="b">…</div>
</div>
```

어디를 고쳐야 하는지 추측해서 답합니다.

구조가 있는 코드를 주면

```
<header>
  <nav>…</nav>
</header>
```

영역을 짚어 정확한 위치를 알려줍니다.

시맨틱 구조는 AI에게 주는 힌트이기도 합니다

오늘 뒤에서 AI에게 리뷰를 받을 때 이 차이가 그대로 드러납니다.

01 · 기준

접근성에는 정해진 기준이 있습니다

WCAG

W3C가 만든 국제 표준입니다.
인식·운용·이해·견고성
네 가지 원칙 아래
지켜야 할 항목이 정리돼 있습니다.

KWCAG

WCAG를 바탕으로 만든
국내 표준입니다.
정식 명칭은 한국형 웹 콘텐츠
접근성 지침입니다.

오늘 배우는 시맨틱 태그와 alt, label은 이 기준에서 가장 먼저 나오는 항목입니다

여기까지 정리

1	**화면은 같다**	시맨틱 태그로 바꿔도 브라우저 화면은 달라지지 않습니다
2	**읽는 쪽이 다르다**	검색엔진, 화면낭독기, AI가 코드를 읽습니다
3	**기준이 있다**	WCAG와 KWCAG에 지켜야 할 항목이 정리돼 있습니다

다음은 실제로 어떤 태그를 쓰는지 봅니다

02

시맨틱 태그로 영역 나누기

여섯 개의 태그 · 구조 설계 · 실습 01

한 페이지를 이렇게 나눕니다

`<header>`	사이트 맨 위, 로고와 제목
`<nav>`	다른 페이지로 가는 메뉴 묶음
`<main>`	이 페이지의 본문, 한 페이지에 하나
`<aside>`	본문 옆의 부가 정보
`<footer>`	맨 아래, 저작권과 연락처

main 안에 들어가는 것

section — 주제로 묶은 한 덩어리
article — 그것만 떼어내도
말이 되는 독립된 글

판단이 서지 않으면

떼어내서 다른 곳에 옮겨도
말이 되면 article,
그렇지 않으면 section입니다.

header — 맨 위의 소개 영역

```
<header>
  <h1>김코딩</h1>
  <p>웹 개발을 배우고 있습니다.</p>
  <nav>…</nav>
</header>
```

무엇이 들어가나요

사이트 이름, 로고, 대표 제목,
그리고 보통 메뉴가 함께
들어갑니다.

한 페이지에 여러 개도 됩니다

article이나 section 안에도
머리말로 쓸 수 있습니다.
오늘은 맨 위 하나만 씁니다.

nav — 다른 곳으로 가는 메뉴 묶음

```
<nav>
  <a href="#intro">소개</a>
  <a href="#work">작업</a>
  <a href="#contact">연락처</a>
</nav>
```

모든 링크를 감싸지는 않습니다

본문 안의 참고 링크까지
nav로 감쌀 필요는 없습니다.
주요 이동 메뉴에만 씁니다.

낭독기가 건너뛰는 지점

nav로 감싸두면 "내비게이션"이라고
알려주고 통째로 건너뛸 수
있게 해줍니다.

main — 이 페이지만의 본문

```
<body>
  <header>…</header>

  <main>
    <!-- 이 페이지에만 있는 내용 -->
  </main>

  <footer>…</footer>
</body>
```

한 페이지에 하나입니다

여러 개를 두면 본문이 어디인지
알 수 없게 됩니다.
오늘 배우는 태그 중 이것만
개수 제한이 있습니다.

header와 footer는 밖에

페이지마다 반복되는 영역이므로
main 밖에 둡니다.

section — 주제로 묶은 한 덩어리

```
<main>
  <section id="intro">
    <h2>소개</h2>
    <p>웹 개발을 배우고 있습니다.</p>
  </section>

  <section id="work">
    <h2>작업</h2>
  </section>
</main>
```

제목을 함께 둡니다

section은 주제로 묶은 덩어리라서
그 주제를 밝히는 제목이 있어야
구분한 의미가 살아납니다.

제목이 없다면

그 묶음은 대부분 section이 아니라
div가 맞습니다. 규칙 위반은 아니고
검사기가 경고로 알려줍니다.

article — 떼어내도 말이 되는 글

```
<section>
  <h2>최근 글</h2>

  <article>
    <h3>첫 번째 글 제목</h3>
    <p>글 내용</p>
  </article>

  <article>…</article>
</section>
```

판단 기준은 하나입니다

떼어내서 다른 곳에 옮겨도
그것만으로 말이 되는가.

이런 것이 article

블로그 글 한 편, 뉴스 기사 하나,
상품 카드 하나, 댓글 하나.

02 · aside·footer

나머지 두 개

aside

```
<aside>
  <h2>이런 글도 있어요</h2>
  <ul>…</ul>
</aside>
```

본문 옆의 부가 정보입니다.
없어도 본문을 읽는 데
지장이 없는 내용입니다.

footer

```
<footer>
  <p>2026 김코딩</p>
</footer>
```

맨 아래 저작권, 연락처,
회사 정보가 들어갑니다.

aside는 위치가 아니라 역할로 정합니다

02 · 구분

그래서 div는 언제 쓰나요

시맨틱 태그를 씁니다

영역에 의미가 있을 때.
메뉴 묶음, 본문, 독립된 글,
머리말과 꼬리말.

div를 씁니다

배치만을 위해 묶을 때.
가운데 정렬하려고 감싸거나,
여러 요소를 한 줄에 놓으려고
묶는 경우.

div를 쓰면 안 된다는 뜻이 아닙니다

의미가 있으면 시맨틱 태그, 배치만 필요하면 div입니다.
전부 시맨틱으로 바꾸는 것이 오히려 흔한 실수입니다.

예전 웹은 표와 div로만 만들었습니다



웹 아카이브에 남아 있는 1990년대 후반 야후 코리아 화면

그때는 태그가 없었습니다

header, nav, main 같은 태그는
HTML5에서 추가됐습니다.
그전에는 table과 div로
배치를 만들었습니다.

지금도 남아 있습니다

오래된 코드를 볼 때
div만 잔뜩 나오는 이유입니다.

되짚기 · 02

지금 우리 코드를 다시 봅니다

```
<body>
  <div class="top">
    <h1>김코딩</h1>
    <div class="menu">…</div>
  </div>

  <div class="body">
    <div id="intro">…</div>
  </div>

  <div class="bottom">…</div>
</body>
```

사람 눈에는 문제가 없습니다

브라우저에서 열면 원하는 대로
잘 보입니다.

그런데 이 문서는

어디가 메뉴이고 어디가 본문인지
코드만 봐서는 알 수 없습니다.
class 이름은 우리끼리 정한
약속일 뿐입니다.

실습 01

div로 만든 페이지를 시맨틱 구조로 바꾸기

Day03 폴더에 아래 about-me.html을 작성합니다.

```
<body>
  <div>
    <h1>김코딩</h1>
    <div>
      <a href="#intro">소개</a>
      <a href="#work">작업</a>
    </div>
  </div>
  <div>
    <div id="intro">
      <h2>소개</h2>
      <img src="me.jpg" alt="프로필 사진">
    </div>
  </div>
  <div><p>2026 김코딩</p></div>
</body>
```

실습 01

div로 만든 페이지를 시맨틱 구조로 바꾸기

시멘틱 태그로 수정해나갑니다. 한 영역 바꿀 때마다 저장하고 새로고침해서 화면이 그대로인지 확인합니다.

```
<body>
  <header>
    <h1>김코딩</h1>
    <nav>
      <a href="#intro">소개</a>
      <a href="#work">작업</a>
    </nav>
  </header>
  <main>
    <section id="intro">
      <h2>소개</h2>
      <img src="me.jpg" alt="프로필 사진">
    </section>
  </main>
  <footer><p>2026 김코딩</p></footer>
</body>
```

바꾸는 순서

- 바깥 영역부터 안쪽으로
- 한 개 바꾸고 새로고침
- 달라지면 바로 되돌리기

확인할 것

- main이 하나만 있다
- header 안에 nav가 있다
- section마다 제목이 있다

실습 01 · 점검

잘 안 되면 여기를 보세요

화면이 달라졌다

닫는 태그가 어긋났습니다.
방금 바꾼 줄부터
다시 봅니다.

무엇으로 바꿀지 모르겠다

그 묶음에 제목이 있는지
먼저 봅니다. 없으면
div가 맞습니다.

section이 너무 많아졌다

배치만을 위한 묶음까지
바꾼 경우입니다.
되돌립니다.

다 한 사람은 Day02의 signup.html도 열어 header와 main을 넣어 보세요

03

제목 계층과 대체 텍스트

h1~h6 설계 · alt 작성 기준

03 · 제목

제목은 크기가 아니라 계층입니다

잘못된 쓰임

```
<h1>내 소개</h1>
<h4>취미</h4>
<h2>연락처</h2>
```

글자 크기를 맞추려고 단계를 건너뛴 경우입니다.

바른 쓰임

```
<h1>내 소개</h1>
<h2>취미</h2>
<h2>연락처</h2>
```

같은 층위의 제목은 같은 단계로 둡니다.

크기는 CSS로 정합니다

h2가 너무 크다면 CSS에서 font-size를 줄이면 됩니다. 그것 때문에 h4를 쓰지 않습니다.

제목만 뽑으면 목차가 되어야 합니다

```
h1  김코딩
  h2  소개
  h2  작업
    h3  개인 소개 페이지
    h3  회원가입 폼
  h2  연락처
```

낭독기가 이 목차로 이동합니다

제목만 훑어 원하는 곳으로
바로 건너뛸 수 있습니다.

단계를 건너뛰면

h2 다음에 h4가 나오면
중간에 무언가 빠진 것처럼
읽힙니다.

h1은 몇 개가 맞나요

```
<!-- 관례 : 페이지에 하나 -->
<h1>김코딩</h1>

<!-- 스펙상 여러 개도 허용됩니다 -->
<section><h1>소개</h1></section>
<section><h1>작업</h1></section>
```

하나로 두는 편이 좋습니다

그 페이지가 무엇에 대한
문서인지 한 줄로 드러납니다.
검사 도구도 이 방식을
권장합니다.

section 안에 넣는다고 제목 단계가 자동으로 조정되지는 않습니다

문서 개요를 자동 계산하는 방식이 한때 제안됐지만 어떤 브라우저에도 구현되지 않았습니다.
h1부터 h6까지 직접 계층에 맞게 적어야 합니다.

alt — 이미지가 보이지 않을 때 대신 읽히는 글

```
<img src="cat.jpg" alt="창가에 앉은 고양이">

<!-- 이럴 때 alt가 대신 나옵니다 -->
· 이미지 파일이 없거나 경로가 틀렸을 때
· 통신이 느려 아직 안 받아졌을 때
· 화면낭독기로 읽을 때
· 검색엔진이 이미지를 이해할 때
```

직접 확인해 보세요

src를 일부러 틀리게 적고
새로고침하면 alt에 적은 글이
그 자리에 나옵니다.

alt는 있으면 좋은 것이 아니라 img에 반드시 있어야 하는 속성입니다

03 · alt 작성

이미지 종류에 따라 다르게 적습니다

내용이 있는 이미지

```
<img src="cat.jpg"
  alt="창가에 앉은
  고양이">
```

무엇이 보이는지 적습니다.

장식용 이미지

```
<img src="line.png"
  alt="">
```

빈 값으로 두면 건너뜁니다.

링크 안의 이미지

```
<a href="/home">
  <img src="logo.png"
  alt="홈으로">
</a>
```

눌렀을 때 할 일을 적습니다.

속성 자체를 빼는 것과 빈 값으로 두는 것은 다릅니다

03 · alt 점검

alt에 넣지 않는 것

이렇게 하면 안 됩니다

```
alt="이미지"
alt="사진"
alt="img_2024_final.jpg"
```

낭독기는 이미 이미지라는 사실을 먼저 알려줍니다.

이렇게 하세요

```
alt="창가에 앉은 고양이"
alt=""
alt="홈으로"
```

무엇이 보이는지, 또는 무슨 일을 하는지 적습니다.

판단이 어려우면 이 문장을 대신 넣어도 페이지가 말이 되는지 읽어봅니다

여기까지 정리

1	**제목은 계층이다**	크기가 아니라 목차 단계에 맞춰 h1부터 순서대로 내려갑니다
2	**h1은 하나로**	스펙상 여러 개도 되지만 하나로 두는 편이 안전합니다
3	**alt는 목적에 따라**	내용이면 설명, 장식이면 빈 값, 링크면 할 일을 적습니다

다음은 폼의 접근성과 AI 리뷰입니다

04

폼 접근성과 AI 리뷰

label·fieldset · 키보드 · 검사기 · 실습 02

04 · label

Day2에서 배운 label을 다시 봅니다

연결되지 않은 경우

```
<label>아이디</label>
<input type="text" name="id">
```

글자와 입력칸이 나란히 있을 뿐
서로 연결돼 있지 않습니다.

연결된 경우

```
<label for="id">아이디</label>
<input type="text" id="id"
  name="id">
```

글자를 눌러도 칸이 선택되고
낭독기가 함께 읽어줍니다.

for와 id가 같아야 연결됩니다

둘 중 하나라도 오타가 있으면 연결이 끊기는데, 화면으로는 알 수 없습니다.

fieldset과 legend — 관련된 입력을 묶습니다

```
<fieldset>
  <legend>요금제 선택</legend>

  <input type="radio" id="free" name="plan">
  <label for="free">무료</label>

  <input type="radio" id="pro" name="plan">
  <label for="pro">유료</label>
</fieldset>
```

legend가 하는 일

낭독기가 각 항목을 읽을 때
"요금제 선택, 무료"처럼
묶음 이름을 함께
알려줍니다.

radio와 checkbox 묶음에 특히 필요합니다

"무료"라는 말만 들으면 무엇에 대한 무료인지 알 수 없기 때문입니다.

04 · placeholder

placeholder는 label을 대신할 수 없습니다

이렇게 하면 안 됩니다

```
<input type="text"
  placeholder="아이디">
```

값을 입력하는 순간 안내가 사라져
무엇을 적는 칸인지
알 수 없게 됩니다.

이렇게 하세요

```
<label for="id">아이디</label>
<input id="id"
  placeholder="영문 4자 이상">
```

label로 이름을 두고
placeholder에는 형식 안내를
적습니다.

AI가 만들어주는 폼 코드에 이 실수가 특히 많습니다

마우스 없이 써 보면 문제가 드러납니다

1	**Tab**	다음 요소로 이동합니다. 순서는 코드에 적힌 순서를 따릅니다
2	**Shift + Tab**	이전 요소로 돌아갑니다
3	**Enter**	링크를 열거나 폼을 전송합니다
4	**Space**	체크박스를 켜고 끕니다

지금 어디에 있는지 보이지 않는다면 그것이 문제입니다

Day6에서 CSS의 focus 표시로 해결합니다. 오늘은 문제를 확인하는 것까지입니다.

04 · AI 리뷰

AI에게 구조를 검토받는 방법

이렇게 물으면 답이 흐립니다

```
이 코드 어때?
```

기준이 없으니 일반적인 답만 돌아옵니다.

기준을 주고 물어봅니다

```
아래 HTML을 시맨틱 구조와
접근성 기준으로 검토해 주세요.
고칠 곳마다 이유를 적어 주세요.
```

확인할 수 있는 답이 돌아옵니다.

무엇을, 어떤 기준으로, 어떤 형식으로 — 세 가지를 함께 적습니다

04 · 검증

받은 답을 그대로 믿지 않습니다

문서에서 확인

제안한 태그나 속성이
실제로 있는지
MDN에서 찾아봅니다.

화면에서 확인

바꾼 뒤 브라우저에서
화면이 의도대로 되는지
직접 봅니다.

말로 설명

왜 그렇게 고쳤는지
한 문장으로 말할 수 있어야
반영합니다.

설명할 수 없는 코드는 넣지 않습니다

지금은 동작하더라도 다음에 문제가 생겼을 때 손을 댈 수 없게 됩니다.

점검

시맨틱에서 자주 나오는 세 가지

!

전부 시맨틱으로 바꿔요

배치만을 위한 묶음까지
section으로 바꾸는 경우입니다.
의미가 없으면 div가 맞습니다.

!

main이 여러 개예요

영역마다 main을 넣으면
본문이 어디인지 알 수 없습니다.
한 페이지에 하나만 둡니다.

!

제목 단계를 건너뛰어요

글자 크기를 맞추려고
h2 다음에 h4를 쓰는 경우입니다.
크기는 CSS로 정합니다.

시맨틱은 태그를 바꾸는 일이 아니라 문서를 정리하는 일입니다

먼저 페이지를 영역으로 나눈 뒤, 각 영역에 맞는 태그를 고르는 순서로 하면 헷갈리지 않습니다.

WRAP UP

오늘 정리

개념

- 같은 화면이라도 코드의 의미는 다르다
- header·nav·main·footer로 영역을 나눈다
- 떼어내도 말이 되면 article이다
- 제목은 크기가 아니라 계층이다
- alt는 이미지의 목적에 따라 달라진다
- 배치만 필요한 묶음에는 div를 쓴다

직접 쓸 수 있어야 하는 것

- div 구조를 시맨틱 태그로 바꾸기
- section에 제목 붙이기
- h1부터 단계에 맞게 제목 적기
- 이미지 종류에 맞는 alt 적기
- fieldset과 legend로 입력 묶기

WRAP UP

한 문장으로 남기면

시맨틱 태그는 화면을 바꾸는 것이 아니라, 이 문서를 읽는 기계에게 구조를 알려주는 일이다.

레퍼런스 W3C WAI — 페이지 구조 튜토리얼

레퍼런스 MDN — 시맨틱이란

레퍼런스 MDN — img 요소와 alt 속성

오늘 정리한 구조는 Day4부터 CSS 선택자의 기준이 되고, Day6에서 다시 점검합니다.

ASSIGNMENT

오늘의 과제

기본과제

개념 확인 5문항

- 시맨틱 태그의 역할 고르기
- section과 article 구분하기
- 제목 계층 오류와 alt 고르기

도전과제

about-me.html 구조 정리하기

div로 되어 있던 영역을 시맨틱 태그로
바꾸고 제목 계층을 정리합니다.
이미지에 alt를 적고 검사기를 돌립니다.
제출물은 파일과 검사 결과 캡처 1장입니다.

제출

노션 과제 페이지 DAY3에 올려주세요. 풀이는 다음 수업을 시작할 때 함께 합니다.

AI TIP · 03

AI에게는 기준을 정해서 물어본다

이렇게 하면 안 됩니다

```
내 HTML 코드 좀 봐줘
```

기준이 없으니 일반적인 답만 돌아옵니다.

이렇게 하세요

```
아래 HTML을 검토해 주세요.
기준은 시맨틱 구조와 접근성입니다.
고칠 곳마다 이유를 함께 적어 주세요.
```

확인할 수 있는 답이 돌아옵니다.

좋은 질문에는 세 가지가 들어 있습니다

① 무엇을 검토할지 ② 어떤 기준으로 볼지 ③ 답을 어떤 형식으로 받을지

오늘의 연습 — 같은 코드를 기준 없이 한 번, 기준을 주고 한 번 물어보고 답을 비교해 보세요.

NEXT

DAY 04

CSS 개념 및 선택자

지금까지는 구조만 만들었고 화면은 브라우저 기본 모양 그대로였습니다.
다음 시간에는 이 구조에 색과 크기를 입히는 CSS를 시작합니다.
오늘 정리한 시맨틱 구조가 그대로 선택자의 기준이 됩니다.

수고하셨습니다.